

ANÁLISIS CRÍTICO COMPARATIVO DE UN SISTEMA DISTRIBUIDO REAL

**Uber: Coordinación en Tiempo Real como Sistema Distribuido
a Escala Global**

Asignatura: Sistemas Distribuidos

Unidad: 1 – Fundamentos de Sistemas Distribuidos

Tipo de actividad: Trabajo Autónomo Sumativo (Individual)

Estudiante:

Luna Mora Ernesto Gregory

Docente:

Guerrero Ulloa Glesiton Ciceron

Universidad Técnica Estatal de Quevedo
Facultad de Ciencias de la Computación y Diseño Digital

2026

Índice

1. Introducción	2
2. Transparencias de Distribución en Uber	2
2.1. Transparencia de Acceso	2
2.2. Transparencia de Ubicación	3
2.3. Transparencia de Migración	3
2.4. Transparencia de Replicación	3
2.5. Transparencia de Concurrencia y Fallos	3
3. Análisis bajo el Teorema CAP	4
3.1. Posición de Uber en el espacio CAP	4
3.2. Decisiones de diseño y compromisos	4
3.3. El modelo PACELC	4
4. Modelos de Comunicación	5
4.1. Comunicación síncrona: RPC y REST	5
4.2. Comunicación asíncrona: Kafka	5
5. Tolerancia a Fallos	5
5.1. Arquitectura de Redundancia y Failover	5
5.2. Mecanismos de Resiliencia Comunes	6
6. Análisis Crítico y Discusión	6
6.1. Fortalezas del Diseño Distribuido	6
6.2. Debilidades y Desafíos	6
7. Conclusiones	6
Repositorio del Proyecto	7

1. Introducción

En la última década, la irrupción de las plataformas de movilidad bajo demanda ha planteado desafíos de ingeniería sin precedentes. Uber, fundada en 2009 en San Francisco, opera actualmente en más de 70 países y gestiona decenas de millones de viajes diarios, lo que la convierte en uno de los sistemas distribuidos a mayor escala del mundo. Su arquitectura debe garantizar la coordinación en tiempo real entre conductores, pasajeros y sistemas de pago, todo ello con latencias de milisegundos y disponibilidad cercana al 100 %.

El presente trabajo analiza de forma crítica la arquitectura distribuida de Uber aplicando los conceptos centrales de la Unidad 1: las **transparencias de distribución** definidas en el modelo de referencia ANSA/ISO, el **teorema CAP** de Brewer, los **modelos de comunicación** presentes en la plataforma y las **estrategias de tolerancia a fallos** implementadas en producción.

Resumen Ejecutivo: Uber opera más de 4.500 microservicios desplegados más de 100.000 veces por semana por aproximadamente 4.000 ingenieros. Su infraestructura combina centros de datos propios con nubes de Oracle y Google Cloud, formando un sistema distribuido activo-activo multi-región que constituye un caso de estudio ideal para los conceptos de la Unidad 1.

El análisis se estructura en cuatro secciones principales que abordan cada concepto teórico con evidencia empírica extraída de la documentación técnica pública del equipo de ingeniería de Uber y artículos académicos publicados por sus propios investigadores.

2. Transparencias de Distribución en Uber

Las transparencias de distribución son mecanismos mediante los cuales un sistema distribuido oculta su naturaleza distribuida al usuario o al desarrollador. El modelo ANSA/ISO identifica varios tipos; a continuación se analiza cómo Uber implementa o negocia cada una.

2.1. Transparencia de Acceso

Uber unifica el acceso a sus recursos a través de un framework propio de llamadas a procedimiento remoto (RPC) denominado **YARPC** (*Yet Another RPC*). Este framework permite que un microservicio invoque a otro sin conocer si el destino se ejecuta en el mismo rack, en otro centro de datos o en una nube pública. Los desarrolladores utilizan una interfaz de programación idéntica independientemente del protocolo subyacente (HTTP/1.1, HTTP/2, TChannel).

2.2. Transparencia de Ubicación

La plataforma **Up** (*Uber Platform*), introducida en 2023, abstrae la ubicación física de los servicios. Los 4.500 microservicios pueden migrarse entre infraestructura propia (on-premise), Oracle Cloud y Google Cloud sin cambios en el código de los clientes. El sistema de descubrimiento de servicios mantiene un anillo hash consistente que redirige las solicitudes al nodo propietario de cada clave sin exponer la topología de red.

2.3. Transparencia de Migración

Durante la migración masiva de microservicios a la nube, Uber diseñó mecanismos de tráfico en paralelo (*shadow traffic*) que redirigían tráfico real hacia los nuevos entornos sin interrupciones para los usuarios finales. Las migraciones entre plataformas de contenedores se ejecutaron de forma transparente a nivel de la aplicación.

2.4. Transparencia de Replicación

Uber emplea **Cassandra** con replicación multi-clúster dentro de cada región: cada escritura de aplicación resulta en escrituras a dos clústeres distintos con tres réplicas cada uno, y replicación asíncrona entre regiones. El cliente nunca gestiona directamente las réplicas; la biblioteca de acceso a datos abstrae por completo este comportamiento.

2.5. Transparencia de Concurrencia y Fallos

La transparencia de concurrencia se implementa mediante serialización a nivel de aplicación: cada clave tiene un proceso propietario único que organiza los ciclos de lectura, modificación y escritura. Por otra parte, la transparencia de fallos permite que los clientes observen respuestas exitosas incluso cuando nodos individuales han fallado, gracias al sistema de cambio automático (*failover*).

Cuadro 1: Resumen de transparencias de distribución en Uber

Transparencia	Mecanismo de Uber	Nivel logrado
Acceso	YARPC (RPC unificado)	Alto
Ubicación	Plataforma Up + Kubernetes	Alto
Migración	Shadow traffic	Medio-Alto
Replicación	Cassandra multi-clúster	Alto
Concurrencia	Ringpop (anillo de hash)	Medio
Fallos	Arquitectura Failover	Alto
Escalado	Autoescalado en Kubernetes	Alto

3. Análisis bajo el Teorema CAP

El teorema CAP establece que un sistema distribuido no puede garantizar simultáneamente **Consistencia** (C), **Disponibilidad** (A) y **Tolerancia a Particiones** (P). Dado que las particiones de red son inevitables en cualquier sistema distribuido real, la elección fundamental durante una falla es priorizar la consistencia o la disponibilidad.

3.1. Posición de Uber en el espacio CAP

Uber adopta principalmente una posición **AP** (Disponibilidad y Tolerancia a Particiones), ofreciendo una consistencia eventual en la mayoría de sus subsistemas. Esta elección es coherente con su modelo de negocio: es preferible mostrar al pasajero la ubicación de un conductor con un retraso de pocos segundos que denegar el servicio o bloquear la pantalla durante una fluctuación de la red.

Cuadro 2: Clasificación de los sistemas en el Teorema CAP

Enfoque	Prioridad en Partición	Ejemplos de Sistemas
CP	Consistencia Estricta (Bloqueo)	Bases de datos bancarias, RDBMS tradicionales
AP (Uber)	Alta Disponibilidad (Datos eventuales)	Plataforma de viajes, Geolocalización

3.2. Decisiones de diseño y compromisos

- **Consistencia eventual en geolocalización:** El servicio de despacho (*dispatch*) tolera que diferentes nodos vean posiciones de conductores ligeramente desactualizadas. El impacto en el usuario es mínimo (retraso de 1 a 3 segundos) pero asegura que la aplicación siga funcionando.
- **Consistencia fuerte en pagos:** A diferencia de la ubicación, el sistema de pagos y facturación emplea consistencia fuerte. Se utiliza un almacenamiento transaccional para garantizar que no ocurran cobros duplicados ni pérdidas de dinero, asumiendo una mayor latencia.
- **Modelo activo-activo multi-región:** Uber opera con un diseño donde cada región geográfica puede absorber el tráfico global ante un fallo crítico regional, priorizando la disponibilidad continua del servicio.

3.3. El modelo PACELC

El modelo PACELC extiende el teorema CAP al evaluar el comportamiento cuando no hay fallas: se debe elegir entre consistencia (C) y latencia (L). Uber se ubica en el cuadrante **PA/EL** (Prioriza Disponibilidad ante particiones, y baja Latencia en funcionamiento normal), sacrificando la consistencia estricta en favor de la rapidez en las actualizaciones de mapas.

4. Modelos de Comunicación

4.1. Comunicación síncrona: RPC y REST

La comunicación directa entre los servicios internos de Uber es mayormente síncrona mediante **YARPC**. Estas llamadas se usan cuando el resultado se necesita de inmediato: por ejemplo, el servicio de emparejamiento (*matching*) entre conductor y pasajero no puede esperar, requiere confirmación instantánea. Para evitar que el sistema se caiga por completo si un servicio responde lento, se usan técnicas básicas como límites de tiempo (*timeouts*) y cancelaciones rápidas (*circuit breakers*).

4.2. Comunicación asíncrona: Kafka

Para eventos que no requieren una respuesta inmediata, Uber utiliza **Apache Kafka** como bus de mensajes distribuido. Los registros de inicio de viaje, los históricos de coordenadas GPS y las notificaciones automáticas se envían como eventos asíncronos. Esto permite que los servicios de análisis de datos procesen la información a su propio ritmo sin ralentizar la aplicación del usuario.

Cuadro 3: Modelos de comunicación en la arquitectura de Uber

Modelo	Tecnología Base	Caso de Uso Principal
RPC Síncrono	YARPC / gRPC	Asignación inmediata de viajes
REST Síncrono	HTTP / JSON	Interfaz con la app móvil cliente
Asíncrono	Apache Kafka	Procesamiento de historial y alertas

5. Tolerancia a Fallos

5.1. Arquitectura de Redundancia y Failover

Uber cuenta con una arquitectura de failover diseñada para cambiar el tráfico de una región a otra de forma eficiente si un centro de datos entero se desconecta. Para gestionar la carga adecuadamente en una emergencia, el sistema clasifica sus componentes en:

1. **Servicios Críticos:** Autenticación, emparejamiento y procesamiento de pagos. No pueden detenerse bajo ninguna circunstancia.
2. **Servicios No Críticos:** Historial de viajes pasados, recomendaciones personalizadas y analíticas. Pueden desactivarse temporalmente para ahorrar recursos durante un fallo.

5.2. Mecanismos de Resiliencia Comunes

Entre las estrategias estándar aplicadas por los ingenieros de Uber para mantener la tolerancia a fallos se encuentran:

- **Aislamiento (Bulkheads):** Se separan los recursos de procesamiento para que un problema en el módulo de opiniones o reseñas no consuma la memoria del módulo de geolocalización.
- **Reintentos con variación (Jitter):** Si una conexión falla, las aplicaciones reintentan la comunicación usando tiempos de espera aleatorios, evitando saturar los servidores recuperados.
- **Pruebas de Caos (Chaos Engineering):** Se simulan caídas de servidores de forma controlada directamente en entornos de prueba para comprobar que el sistema distribuido reaccione automáticamente sin intervención humana.

6. Análisis Crítico y Discusión

6.1. Fortalezas del Diseño Distribuido

La modularidad en miles de microservicios permite actualizaciones constantes sin detener la plataforma completa. Asimismo, la decisión de priorizar la disponibilidad (modelo AP) es correcta para las necesidades reales del transporte. Una aplicación rígida que se colgara por buscar una consistencia de datos perfecta no sería competitiva en el mercado actual.

6.2. Debilidades y Desafíos

Por otro lado, la consistencia eventual puede causar desajustes temporales en zonas de alta densidad (como aeropuertos o estadios), donde múltiples usuarios intentan tomar el mismo vehículo y la información de ubicación tarda segundos en sincronizarse. Adicionalmente, coordinar y monitorear contratos entre más de 4.000 servicios genera una complejidad operativa inmensa para los equipos de ingeniería.

7. Conclusiones

El estudio de un caso real como Uber demuestra que los conceptos fundamentales de los sistemas distribuidos no son teorías aisladas, sino decisiones prácticas de diseño:

- Las transparencias de distribución se logran mediante capas de abstracción bien definidas que ocultan la infraestructura física a los ingenieros de software.
- No hay una solución única para el dilema del Teorema CAP; los sistemas complejos combinan consistencia fuerte (para dinero y transacciones) con consistencia eventual (para flujos de coordenadas en tiempo real).

- La tolerancia a fallos requiere un diseño inteligente que aprenda a priorizar módulos críticos sobre funciones secundarias cuando los recursos se ven limitados por contingencias de red.

Repositorio del Proyecto

El código fuente, archivos y recursos relacionados con este trabajo se encuentran disponibles en el siguiente repositorio público de GitHub:

Nombre del repositorio: Aplicaciones Distribuidas – Luna Mora Ernesto Gregory

Enlace: <https://github.com/Ernesto835/Aplicaciones-distribuidas.git>

Referencias

- [1] Uber Engineering (2023). *Up: Portable Microservices Ready for the Cloud*. Uber Blog. Disponible en: <https://eng.uber.com/up-portable-microservices-ready-for-the-cloud/>
- [2] Uber Engineering (2015). *Service-Oriented Architecture: Scaling the Uber Engineering Codebase*. Uber Blog. Disponible en: <https://www.uber.com/us/en/blog/service-oriented-architecture/>
- [3] Uber Engineering (2021). *Uber's Fulfillment Platform: Ground-up Re-architecture*. Uber Blog. Disponible en: <https://www.uber.com/us/en/blog/fulfillment-platform-rearchitecture/>
- [4] Bansal, M., Chabbi, M., et al. (2024). *Uber's Failover Architecture: Reconciling Reliability and Efficiency*. arXiv preprint arXiv:2603.07345. Disponible en: <https://arxiv.org/abs/2603.07345>
- [5] Tanenbaum, A. S., & Van Steen, M. (2017). *Distributed Systems: Principles and Paradigms* (3.^a ed.). Pearson.
- [6] Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- [7] Luna Mora, E. G. (2026). *Repositorio del trabajo: Aplicaciones Distribuidas*. GitHub. Disponible en: <https://github.com/Ernesto835/Aplicaciones-distribuidas.git>